

All sections covered	Exam Format
Section 1 - Design Applications Section 2 - Data Preparation Section 3 - Application Development Section 4 - Assembling and Deploying Apps Section 5 - Governance Section 6 - Evaluation and Monitoring	Total Number of Questions: 45 Question Types: Multiple Select, Multiple Choice Time Limit: 90 minutes Languages: English Passing Score: 70%

Question 1

A Generative AI Engineer is tasked with enhancing a RAG application that uses customer transaction data stored in Delta tables in the production environment. The team needs to test changes in the development environment using realistic data without compromising security or affecting production performance. What is the best approach in Databricks?

1. Use a synthetic dataset generated within Databricks that mimics the schema and general characteristics of production data.

Explanation

Incorrect: Synthetic data may not capture the nuances of real-world data, leading to less reliable testing.

2. Copy the entire Delta table into the development environment without any modifications.

Explanation

Incorrect: Copying data without anonymization risks exposing sensitive information and increases storage and cost overhead.

3. Use Databricks' Delta Sharing to provide limited query access to the production Delta table from the development environment.

Explanation

Incorrect: While Delta Sharing enables secure sharing, it is better suited for external partners or collaborators, not internal development workflows.

4. Use Databricks' Unity Catalog to create a development-specific view of the production data with access controls and data masking for sensitive fields.

Explanation

Correct: Unity Catalog supports creating secure views and masking rules, allowing development to access realistic data safely without compromising sensitive information.

5. Point the development environment directly to the production Delta table for real-time testing.

Explanation

Incorrect: Direct access to production risks affecting live performance and compromising sensitive data.

Overall explanation

Using Unity Catalog to create a masked view of production data ensures the development environment is safe, secure, and compliant while providing realistic data for testing.

Question 2

A Generative AI Engineer is building a RAG application that answers questions about internal documents for SnoPen AI. The source documents may contain irrelevant content, such as ads or non-company-specific information. Which approach is advisable to filter irrelevant information?

1. Keep all articles because the RAG application needs to understand non-company content to avoid answering questions about them.

Explanation

Incorrect: Retaining irrelevant documents increases noise and reduces retrieval quality.

2. Include in the system prompt that the application is not supposed to answer any questions unrelated to SnoPen AI.

Explanation

Correct: This ensures that answers are scoped to company-specific content.

3. Include in the system prompt that any information it sees will be about SnoPen AI, even if no data filtering is performed.

Explanation

Incorrect: A system prompt cannot compensate for a lack of proper document filtering.

4. Consolidate all SnoPen AI-related documents into a single chunk in the vector database.

Explanation

Incorrect: Consolidating documents reduces granularity and retrieval precision.

Overall explanation

Including constraints in the system prompt ensures the application focuses solely on SnoPen AI-related questions.

Question 3

A Generative AI Engineer must evaluate prompt-response pairs for a customer support chatbot. What criteria are most important to ensure alignment with the model's task? **(Choose two)**

1. Prompt relevance to the customer's query.

Explanation

Correct: Relevant prompts ensure the system generates meaningful responses.

2. Random sampling of responses for variety.

Explanation

Incorrect: Random sampling does not guarantee alignment or quality.

3. Token usage efficiency to reduce computational costs.

Explanation

Incorrect: While important for cost, token usage does not ensure alignment.

4. Prompt verbosity to provide context.

Explanation

Incorrect: Verbosity can dilute the focus of the query.

5. Response completeness, addressing all aspects of the query.

Explanation

Correct: Complete responses improve the chatbot's usefulness and user satisfaction.

Overall explanation

Relevant prompts and complete responses ensure the chatbot aligns with its task, providing accurate and helpful outputs.

Question 4

A Generative AI Engineer is designing an LLM-powered live sports commentary platform that provides real-time updates and analyses. This platform aims to generate summaries based on live game scores, rather than relying on potentially outdated articles. Which tool will enable access to real-time data for generating these game analyses?

1. AutoML

Explanation

Incorrect: AutoML is used for training and deploying models but does not focus on real-time data serving.

2. Foundation Model APIs

Explanation

Incorrect: Foundation Model APIs provide LLM capabilities but do not ingest or serve real-time data.

3. DatabricksIQ

Explanation

Incorrect: DatabricksIQ does not specialize in real-time data ingestion or serving for applications.

4. Feature Serving

Explanation

Correct: Feature Serving enables real-time access to data, making it suitable for live commentary and analyses.

Overall explanation

Feature Serving is the ideal tool for accessing real-time data, ensuring the platform provides up-to-date analyses for users.

Question 5

A Generative AI Engineer is creating an application requiring up-to-date news articles and stock prices. How should they architect the system?

1. Create an agent with tools for SQL querying of Delta tables and web searching. Provide retrieved values to the LLM for response generation.

Explanation

Correct: An agent system with tools for SQL querying and web searching efficiently integrates structured and unstructured data retrieval for real-time responses.

2. Download and store news articles and stock prices in a vector store. Use a RAG architecture to retrieve and generate responses at runtime.

Explanation

Incorrect: RAG is effective for retrieval but does not incorporate real-time updates for stock prices.

3. Use an LLM to summarize news articles and lookup stock tickers to find stock prices.

Explanation

Incorrect: This process does not efficiently integrate data retrieval and generation.

4. Query the Delta table for stock prices and use an LLM to generate search queries for stock volatility analysis.

Explanation

Incorrect: Generating search queries does not streamline data retrieval and response generation.

Overall explanation

An agent-based architecture combining SQL queries and web search ensures real-time integration of stock prices and news for comprehensive responses.

Question 6

A Generative AI Engineer interfaces with an LLM designed to output “In Stock” if the product is available or “Out of Stock” if not. The engineer needs to implement a prompt to format the output correctly for call classification labels. Which prompt is most suitable?

1. Respond with “Out of Stock” if the customer asks for a product.

Explanation

Incorrect: This ignores the possibility of the product being available and does not address the full classification task.

2. Respond with “In Stock” if the customer asks for a product.

Explanation

Incorrect: This lacks specific instructions on how to determine availability or structure the response.

3. You will be given a customer call transcript where the customer asks about product availability. The outputs are either “In Stock” or “Out of Stock.” Format the output in JSON, for example: {“call_id”: “123”, “label”: “In Stock”}.

Explanation

Correct: This prompt provides clear instructions and the desired output format, ensuring consistency and accuracy for call classification.

4. You will be given a customer call transcript where the customer inquires about product availability. Respond with “In Stock” if the product is available or “Out of Stock” if not.

Explanation

Incorrect: While clear, this prompt does not format the output in the desired JSON structure.

Overall explanation

The second prompt includes both clear instructions and the required JSON format, ensuring the output meets classification needs.

Question 7

A Generative AI Engineer is tasked with developing a preprocessing pipeline for a custom LLM application that normalizes user queries and removes unnecessary symbols. Which approach should they use?

1. Use a Spark job to normalize and preprocess user queries.

Explanation

Incorrect: While Spark is powerful, it is overkill for lightweight preprocessing tasks and adds unnecessary complexity.

2. Fine-tune the LLM to preprocess the user queries internally.

Explanation

Incorrect: Fine-tuning the LLM for preprocessing is resource-intensive and not practical compared to using PyFunc.

3. Use a Delta Live Table to preprocess user queries before passing them to the LLM.

Explanation

Incorrect: Delta Live Tables are designed for structured data pipelines, not for preprocessing user queries in real time.

4. Write a PyFunc model to implement the preprocessing logic and integrate it with the LLM application.

Explanation

Correct: PyFunc models allow for modular and reusable custom Python-based preprocessing steps, making them ideal for query normalization.

Overall explanation

PyFunc is an MLflow feature that enables custom preprocessing or postprocessing logic to be easily integrated with machine learning workflows. It is efficient, reusable, and ideal for query normalization tasks.

Question 8

A Generative AI Engineer is building a Generative AI system to recommend the best-matched team member for newly scoped projects. The match must consider project date availability and profile alignment with the project scope. Both the employee profile and project scope are stored as unstructured text. What system architecture should they use?

1. Create a tool for finding available team members given project dates. Embed all project scopes into a vector store, perform a retrieval using team member profiles to find the best team member.

Explanation

Incorrect: Embedding project scopes for retrieval using profiles is less effective than embedding team profiles and filtering based on project scope.

2. Create a tool for finding available team members given project dates. Embed team profiles into a vector store and use the project scope and filtering to perform retrieval to find the available best-matched team members.

Explanation

Correct: Embedding team profiles in a vector store enables semantic matching, and retrieval ensures scalability and precision.

3. Create a tool to find available team members given project dates. Create a second tool that can calculate a similarity score for a combination of team member profile and the project scope. Iterate through the team members and rank by best score to select a team member.

Explanation

Incorrect: This approach lacks scalability and efficiency, especially for large teams.

4. Create a tool for finding team member availability given project dates and another tool that uses an LLM to extract keywords from project scopes. Iterate through available team members' profiles and perform keyword matching to find the best available team member.

Explanation

Incorrect: Keyword matching is simplistic and cannot handle the complexity of unstructured text.

Overall explanation

Embedding team profiles into a vector store and using retrieval ensures precise and scalable matching for large datasets of team members and project scopes.

Question 9

A Generative AI Engineer is tasked with deploying a RAG application for customer support that integrates LLMs and a vector store. The application must comply with data privacy regulations. What compliance measures are critical? **(Choose two)**

1. Use public vector stores to simplify storage.

Explanation

Incorrect: Public vector stores may violate data privacy regulations by exposing sensitive data.

2. Encrypt data at rest and in transit to protect sensitive information.

Explanation

Correct: Encryption ensures data security, meeting privacy regulations.

3. Skip encryption and rely on private network access.

Explanation

Incorrect: Private networks alone do not guarantee data security or compliance.

4. Allow unrestricted access to the application during initial deployment.

Explanation

Incorrect: Unrestricted access compromises compliance and security.

5. Use API key authentication to control access to the application.

Explanation

Correct: API key authentication ensures that only authorized users access the system.

Overall explanation

Encryption and API key authentication are essential for ensuring compliance and security in customer support applications.

Question 10

What is the most suitable library for building a multi-step LLM-based workflow?

1. PySpark

Explanation

Incorrect: PySpark is a distributed computing framework and not intended for LLM-based workflow development.

2. LangChain

Explanation

Correct: LangChain is designed for building multi-step workflows involving LLMs, including tools for prompts, chains, and agents.

3. TensorFlow

Explanation

Incorrect: TensorFlow is a deep learning framework but not specific to multi-step LLM-based workflows.

4. Pandas

Explanation

Incorrect: Pandas is used for data manipulation and analysis but does not support building LLM workflows.

Overall explanation

LangChain provides the tools needed for creating complex workflows with LLMs, making it the most suitable library for this task.

Question 11

A Generative AI Engineer is tasked with developing a RAG application for a small internal group of experts at their company. The application must answer sensitive and confidential questions using an internal knowledge base. High-quality answers are a priority, while latency and throughput are not concerns due to the small user group. Regulatory requirements prohibit transmitting any information to third parties. Which model meets all the requirements?

1. OpenAI GPT-4

Explanation

Incorrect: While GPT-4 provides exceptional quality, it requires external API access, which violates the regulatory requirement of keeping data entirely internal.

2. Dolly 1.5B

Explanation

Incorrect: Dolly is an open-source model that ensures data confidentiality, but its quality is limited compared to larger models like Llama2-70B, making it unsuitable for high-quality, expert-level answers.

3. Llama2-70B

Explanation

Correct: Llama2-70B is an open-source, high-quality model that supports on-premises deployment, ensuring data confidentiality while delivering the best possible quality for sensitive, expert-level queries.

4. BGE-large

Explanation

Incorrect: BGE-large is primarily an embedding model designed for semantic search and retrieval, not generating high-quality, detailed answers.

Overall explanation

Llama2-70B is the best choice for this scenario, meeting the requirements of high-quality answers, data confidentiality, and regulatory compliance. It is open-source, deployable internally, and suited for expert-level applications.

Question 12

A Generative AI Engineer is tasked with designing a reasoning system that evaluates legal contracts for potential risks. The system must highlight risky clauses, propose alternative language, and summarize key terms. What tools should be included? **(Choose two)**

1. An embedding model to semantically analyse contract clauses.

Explanation

Correct: Embeddings help understand clause context and identify risks.

2. A visualization tool to highlight risky clauses interactively.

Explanation

Incorrect: Visualization improves usability but does not aid risk evaluation or proposals.

3. A decision tree to guide users through contract sections.

Explanation

Incorrect: Decision trees are too rigid for dynamic risk evaluation and proposals.

4. A classification model to categorize clauses by risk level.

Explanation

Incorrect: Categorization alone does not provide alternative language or summaries.

5. A retrieval system to fetch legal precedents or alternative clause templates.

Explanation

Correct: Retrievers provide supporting content for proposed changes.

Overall explanation

Embeddings and retrieval systems enable dynamic risk evaluation, clause suggestions, and contract summarization effectively.

Question 13

A Generative AI Engineer is building a system that generates poem-like article summaries (e.g., haikus). The output doesn't match the desired tone or style. Which approach will NOT improve the response?

1. Fine-tune the LLM on a dataset of desired tone and style.

Explanation

Incorrect: Fine-tuning tailors the model to consistently generate text in the desired tone or style.

2. Include few-shot examples in the prompt to the LLM.

Explanation

Incorrect: Few-shot examples train the LLM to emulate the desired style.

3. Provide the LLM with a prompt that explicitly instructs it to generate text in the desired tone and style.

Explanation

Incorrect: A clear prompt can significantly improve the generated tone and style.

4. Use a neutralizer to normalize the tone and style of the underlying documents.

Explanation

Correct: Neutralizing input documents does not directly address the LLM's output tone or style.

Overall explanation

Neutralizing input documents affects their content but does not align the LLM's output tone or style with the desired result.

Question 14

A Generative AI Engineer is tasked with deploying an application that takes advantage of a custom MLflow Pyfunc model to return some interim results. How should they configure the endpoint to pass the secrets and credentials?

1. Use `spark.conf.set()`

Explanation

Incorrect: `spark.conf.set()` is typically used for Spark configuration and is not suitable for securely passing secrets or credentials.

2. Pass variables using the Databricks Feature Store API

Explanation

Incorrect: The Feature Store API is used for managing and retrieving feature data, not for passing secrets or credentials.

3. Pass the secrets in plain text

Explanation

Incorrect: Passing secrets in plain text poses significant security risks and is not recommended.

4. Add credentials using environment variables

Explanation

Correct: Environment variables are a secure and scalable way to pass secrets or credentials to an application.

Overall explanation

Environment variables ensure secure handling of secrets or credentials without exposing them in the code.

Question 15

A Generative AI Engineer is developing a RAG application for a large dataset of customer support tickets. However, the application is producing incomplete responses, where the answers are cut off or lack sufficient detail. What strategy should the engineer apply to mitigate this issue? **(Choose two)**

1. Increase the chunk size of documents to reduce fragmentation of context.

Explanation

Incorrect: Increasing chunk size risks exceeding token limits for the LLM, worsening the issue of incomplete responses.

2. Add segment labels (e.g., section headers) to the chunks to improve context relevance during retrieval.

Explanation

Correct: Including segment labels enhances the LLM's understanding of the chunked context, reducing the chances of incomplete or irrelevant responses.

3. Limit the number of retrieved chunks to minimize response errors.

Explanation

Incorrect: Reducing the number of retrieved chunks may lead to incomplete context, exacerbating the issue.

4. Use a larger embedding model to improve the quality of embeddings and document retrieval.

Explanation

Incorrect: While a larger embedding model can enhance retrieval quality, it does not directly address incomplete responses caused by context limitations.

5. Decrease the chunk size of documents to ensure more manageable context for the LLM.

Explanation

Correct: Decreasing chunk size ensures the LLM can fully process and retrieve relevant chunks within its context window, improving response completeness.

Overall explanation

Decreasing the chunk size ensures that context fits within the LLM's token limit, while adding segment labels provides clearer structure for the model to generate complete responses.

Question 16

A Generative AI Engineer is developing a fraud detection system for an e-commerce platform. The system uses an LLM to flag suspicious transactions based on textual descriptions, customer history, and transaction metadata. The engineer wants to track multiple model experiments, record performance metrics, and ensure the selected model can be easily deployed for production. Which approach should they use?

1. Write custom Python scripts to store experiment logs in a local file system and deploy the model using Flask.

Explanation

Incorrect: Custom scripts lack the robustness and scalability provided by MLflow for tracking and deployment.

2. Use a cloud-based document management system to track experiments and deployment steps.

Explanation

Incorrect: Cloud-based document systems are not designed for tracking machine learning experiments or deployments.

3. Use a spreadsheet to manually record experiment results and maintain a log of model configurations.

Explanation

Incorrect: Manual tracking is error-prone, inefficient, and unsuitable for production-grade workflows.

4. Use MLflow to log model experiments, track performance metrics, and streamline deployment to production.

Explanation

Correct: MLflow is designed for managing the machine learning lifecycle, allowing experiment tracking, artifact logging, and easy deployment.

5. Use TensorFlow's built-in logging and deployment tools.

Explanation

Incorrect: TensorFlow's tools are useful for deep learning models but lack the general-purpose lifecycle management capabilities of MLflow.

Overall explanation

MLflow provides a complete solution for tracking experiments, logging performance metrics, and deploying the selected model, making it ideal for building and deploying machine learning systems efficiently.

Question 17

A Generative AI Engineer is developing an LLM application that users can use to generate personalized birthday poems based on their names. Which technique would be most effective in safeguarding the application, given the potential for malicious user inputs?

1. Increase the amount of compute that powers the LLM to process input faster

Explanation

Incorrect: Increasing compute power addresses performance, not security concerns related to malicious inputs.

2. Implement a safety filter that detects any harmful inputs and ask the LLM to respond that it is unable to assist

Explanation

Correct: Safety filters are designed to detect and handle harmful inputs, ensuring the application remains secure and responsible.

3. Ask the LLM to remind the user that the input is malicious but continue the conversation with the user

Explanation

Incorrect: Continuing the conversation after identifying malicious inputs could lead to further security risks.

4. Reduce the time that the users can interact with the LLM

Explanation

Incorrect: Reducing interaction time does not mitigate malicious inputs or safeguard the application effectively.

Overall explanation

Safety filters detect harmful inputs and prevent the LLM from processing or responding to them, ensuring secure operation.

Question 18

A Generative AI Engineer is testing a simple prompt template in LangChain using the code below but is getting an error: *from langchain.chains import LLMChain from langchain_community.llms import OpenAI from langchain_core.prompts import PromptTemplate prompt_template = "Tell me a {adjective} joke" prompt = PromptTemplate(input_variables=["adjective"], template=prompt_template) llm = LLMChain(prompt=prompt) llm.generate([{"adjective": "funny"}])* Assuming the API key was properly defined, what change does the Generative AI Engineer need to make to fix their chain?

1. `prompt_template = "Tell me a {adjective} joke" prompt = PromptTemplate( input_variables=["adjective"], template=prompt_template ) llm = LLMChain(prompt=prompt) llm.generate("funny")`

Explanation

This fails because the generate method here improperly calls "funny" directly instead of mapping it to the input variable. It also does not address the root issue with the chain initialization.

2. `prompt_template = "Tell me a {adjective} joke" prompt = PromptTemplate( input_variables=["adjective"], template=prompt_template ) llm = LLMChain(prompt=prompt.format("funny")) llm.generate()`

Explanation

This involves formatting the template manually, which circumvents the structured prompt framework of LangChain. This is a hack, not a fix.

3. `prompt_template = "Tell me a {adjective} joke" prompt = PromptTemplate( input_variables=["adjective"], template=prompt_template ) llm = LLMChain(llm=OpenAI(), prompt=prompt) llm.generate([{"adjective": "funny"}])`

Explanation

This is the correct answer because the LLMChain must be initialized with both the PromptTemplate and the LLM instance (OpenAI()). This ensures the chain is connected to the language model for proper execution.

4. `prompt_template = "Tell me a {adjective} joke" prompt = PromptTemplate( input_variables=["adjective"], template=prompt_template, llm=OpenAI() ) llm = LLMChain(prompt=prompt) llm.generate([{"adjective": "funny"}])`

Explanation

The PromptTemplate cannot accept an LLM instance (e.g., OpenAI()) as an argument, so this will result in another error.

Overall explanation

The error occurs because the LLMChain is not properly connected to an LLM instance, such as OpenAI(). In LangChain, the PromptTemplate defines the structure of the prompt and its input variables, while the LLMChain links this template to the LLM to enable output generation. Without an LLM instance, the chain cannot process prompts or produce responses, leading to failure. The correct solution involves initializing the LLMChain with both the PromptTemplate and the LLM. Other approaches fail because they either omit the required LLM connection, misplace components, or misuse the prompt formatting, which does not address the root issue.

Question 19

A Generative AI Engineer must evaluate the suitability of a model for summarizing academic research papers. The evaluation should consider semantic accuracy and domain relevance. What methods should they prioritize? **(Choose two)**

1. Perform human evaluation to assess the summaries for domain correctness.

Explanation

Correct: Human evaluation captures nuanced insights that automated metrics may miss.

2. Use ROUGE for content overlap and domain-specific metrics for semantic accuracy.

Explanation

Correct: ROUGE evaluates content overlap, while domain-specific metrics ensure relevance and semantic fidelity.

3. Measure perplexity to evaluate fluency.

Explanation

Incorrect: Perplexity assesses language fluency but not semantic accuracy or domain relevance.

4. Evaluate token usage to optimize cost.

Explanation

Incorrect: Token usage is unrelated to relevance or semantic accuracy.

5. Use BLEU to measure similarity with reference texts.

Explanation

Incorrect: BLEU is more suited to translation tasks, not academic summarization.

Overall explanation

Combining ROUGE with human evaluation ensures accurate and relevant summaries tailored to academic research.

Question 20

A Generative AI Engineer received the following business requirements for an external chatbot: The chatbot needs to know what types of questions the user asks and routes to appropriate models to answer the questions. For example, the user might ask about upcoming event details or ticket purchases. What is an ideal workflow for such a chatbot?

1. The chatbot should only process payments.

Explanation

Incorrect: This ignores the broader scope of user queries.

2. There should be two different chatbots handling different types of user queries.

Explanation

Incorrect: Splitting functionality into separate chatbots is inefficient and increases complexity.

3. The chatbot should only look at previous event information.

Explanation

Incorrect: Limiting the chatbot to event history neglects other query types, such as ticket purchases.

4. The chatbot should be implemented as a multi-step LLM workflow. First, identify the type of question asked, then route the question to the appropriate model. If it's an upcoming event question, send the query to a text-to-SQL model. If it's about ticket purchasing, the customer should be redirected to a payment platform.

Explanation

Correct: A multi-step LLM workflow ensures flexibility and efficiency in handling diverse user queries.

Overall explanation

A multi-step LLM workflow allows the chatbot to handle diverse queries efficiently by identifying the query type and routing it appropriately.

Question 21

A Generative AI Engineer is creating an LLM system that retrieves news articles from the year 1918 related to a user query and summarizes them. However, the summaries often include explanations of how they were generated, which is undesirable. Which change should the engineer implement to resolve this?

1. Split the LLM output by newline characters to truncate away the summarization explanation.

Explanation

Incorrect: Splitting the output by newlines may cut off critical parts of the summary and does not address the root issue.

2. Revisit their document ingestion logic, ensuring that the news articles are being ingested properly.

Explanation

Incorrect: Ingestion logic determines input quality but is not responsible for undesired output explanations.

3. Provide few-shot examples of desired output format to the system and/or user prompt.

Explanation

Correct: Few-shot examples guide the LLM to produce only the desired summary format, mitigating undesired explanations.

4. Tune the chunk size of news articles or experiment with different embedding models.

Explanation

Incorrect: Chunk size or embedding models impact retrieval quality but do not influence output formatting.

Overall explanation

Providing few-shot examples demonstrates the correct format, ensuring the LLM outputs concise summaries without additional explanations.

Question 22

A Generative AI Engineer is designing a chatbot for a gaming company to engage users during online gameplay. Which metric would help increase engagement and retention?

1. Randomness

Explanation

Incorrect: Randomness may confuse users and reduce engagement.

2. Repetition of responses

Explanation

Incorrect: Repetitive responses bore users and lower retention.

3. Lack of relevance

Explanation

Incorrect: Irrelevant responses frustrate users and reduce engagement.

4. Diversity of responses

Explanation

Correct: Diverse responses maintain user interest and increase retention.

Overall explanation

Diverse responses keep the chatbot engaging and help maintain user interest, improving retention during gameplay.

Question 23

A Generative AI Engineer has trained an LLM on Databricks and wants to deploy it. What is the easiest process for deploying the model on Databricks?

1. Log the model using MLflow during training, directly register it to Unity Catalog using the MLflow API, and start a serving endpoint.

Explanation

Correct: This is the standard and easiest process for deploying a model on Databricks.

2. Wrap the LLM into a Flask app and serve using Gunicorn.

Explanation

Incorrect: Flask/Gunicorn setups are unnecessary for Databricks, which provides managed serving endpoints.

3. Log the model as a pickle object, upload to Unity Catalog Volume, register it with MLflow, and start a serving endpoint.

Explanation

Incorrect: Pickle is not the standard format for deploying models on Databricks.

4. Save the model locally, build a Docker image, and run a Docker container.

Explanation

Incorrect: While functional, this process is more complex than using MLflow.

Overall explanation

MLflow simplifies model deployment on Databricks by managing registration and serving processes seamlessly.

Question 24

A Generative AI Engineer is building a system which will answer questions on latest stock news articles. Which will NOT help with ensuring the outputs are relevant to financial news?

1. Incorporate manual reviews to correct any problematic outputs prior to sending to the users.

Explanation

Incorrect: Manual reviews improve relevance by ensuring outputs are specific and appropriate.

2. Implement a profanity filter to screen out offensive language.

Explanation

Correct: A profanity filter does not enhance relevance to financial news, as it is focused on language safety rather than domain-specific content.

3. Implement a comprehensive guardrail framework that includes policies for content filters tailored to the finance sector.

Explanation

Incorrect: Tailored content filters ensure relevance by filtering out inappropriate or irrelevant content.

4. Increase the compute to improve processing speed of questions to allow greater relevancy analysis.

Explanation

Incorrect: While compute speed improves performance, it does not directly impact content relevance.

Overall explanation

A profanity filter does not affect the relevance of financial news outputs, as it addresses only the appropriateness of language.

Question 25

A Generative AI Engineer created a RAG application for employees to retrieve answers from an internal knowledge base. Now they want to formally evaluate the system's performance. How should they evaluate the system?

1. Use cosine similarity score to comprehensively evaluate the quality of the final generated answers.

Explanation

Incorrect: Cosine similarity measures retrieval quality but does not fully evaluate the generated answers.

2. Benchmark multiple LLMs with the same data and pick the best LLM for the job.

Explanation

Incorrect: Benchmarking LLMs does not address retrieval-specific performance.

3. Curate a dataset to test retrieval and generation separately. Use MLflow's built-in evaluation metrics for evaluation.

Explanation

Correct: Testing retrieval and generation separately ensures comprehensive evaluation of both components.

4. Use an LLM-as-a-judge to evaluate the quality of the final answers generated.

Explanation

Incorrect: LLM-as-a-judge evaluation is subjective and less reliable than curated datasets.

Overall explanation

Separately evaluating retrieval and generation with curated datasets and MLflow metrics provides a systematic approach to improving performance.

Question 26

A small and cost-conscious startup in cancer research wants to build a RAG application using Foundation Model APIs. How can they build a quality application while remaining cost-efficient?

1. Limit the number of queries a customer can send per day.

Explanation

Incorrect: This reduces usage but does not improve the application's quality-cost balance.

2. Limit the number of relevant documents available for retrieval.

Explanation

Incorrect: Reducing document availability may lower the application's quality.

3. Pick a smaller LLM that is domain specific.

Explanation

Correct: Smaller, domain-specific LLMs reduce costs while maintaining relevance and accuracy.

4. Use the largest LLM for best performance.

Explanation

Incorrect: Larger models increase costs unnecessarily for the startup's needs.

Overall explanation

Smaller, domain-specific LLMs strike a balance between quality and cost, catering to the startup's specific needs.

Question 27

A Generative AI Engineer wants to build an LLM-based solution to help a restaurant improve its online customer experience with bookings by automatically handling common customer inquiries. The goal of the solution is to minimize escalations to human intervention and phone calls while maintaining a personalized interaction. To design the solution, the Generative AI Engineer needs to define the input data to the LLM and the task it should perform. Which input/output pair will support their goal?

1. **Input:** Online chat logs; **Output:** Cancellation options

Explanation

Generating cancellation options based on online chat logs as input does not directly support the goal of minimizing escalations to human intervention and phone calls while maintaining personalized interactions. While providing cancellation options is important, the primary focus should be on automating responses to common inquiries related to bookings.

2. Input: Online chat logs; Output: Buttons that represent choices for booking details

Explanation

Using online chat logs as input to generate buttons representing choices for booking details aligns with the goal of minimizing escalations to human intervention and phone calls while maintaining personalized interactions. By providing predefined choices, the solution can automate responses to common inquiries and guide customers through the booking process.

3. **Input:** Online chat logs; **Output:** Group the chat logs by users, followed by summarizing each user's interactions

Explanation

Grouping chat logs by users and summarizing interactions does not directly support the goal of minimizing escalations to human intervention and phone calls while maintaining personalized interactions. This task focuses more on organizing data rather than providing automated responses to common inquiries.

4. **Input:** Customer reviews; **Output:** Classify review sentiment

Explanation

Classifying review sentiment based on customer reviews is not directly related to the goal of minimizing escalations to human intervention and phone calls while maintaining personalized interactions in the context of improving online customer experience with bookings. This task focuses on sentiment analysis rather than providing automated responses to inquiries.

Overall explanation

Generating interactive options for booking inquiries aligns with the goal of minimizing escalations while maintaining personalization.

Question 28

A Generative AI Engineer is tasked with extracting content from a set of HTML-based user manuals. The manuals contain product details in nested tables. What is the most suitable tool for text extraction? **(Choose two)**

1. pdfplumber for extracting table data.

Explanation

Incorrect: pdfplumber is designed for PDFs, not HTML.

2. pytesseract for OCR-based text extraction.

Explanation

Incorrect: pyesseract is unnecessary for extracting text from HTML files.

3. PyPDF2 for reading text from user manuals.

Explanation

Incorrect: PyPDF2 does not parse HTML content.

4. LXML for faster parsing and processing of large HTML documents.

Explanation

Correct: LXML enhances efficiency for large-scale HTML processing.

5. BeautifulSoup for parsing HTML and extracting text from structured elements.

Explanation

Correct: BeautifulSoup is ideal for navigating and extracting structured content from HTML.

Overall explanation

Combining BeautifulSoup and LXML ensures robust and efficient extraction of structured text from HTML-based manuals.

Question 29

A Generative AI Engineer is tasked with chunking and indexing legal texts for a RAG application. The legal texts contain lengthy sections; some exceeding token limits. What is the best chunking strategy?

1. Split the text into logical sections based on headings and subheadings, ensuring chunks remain under token limits.

Explanation

Correct: Logical chunking preserves context and ensures compliance with token constraints.

2. Process each document as a single chunk.

Explanation

Incorrect: Single chunks exceeding token limits are infeasible for retrieval and processing.

3. Split the text into equal-sized chunks without considering structure.

Explanation

Incorrect: Equal-sized chunks risk breaking meaningful sections, reducing relevance.

4. Overlap all chunks by 30% to ensure context preservation.

Explanation

Incorrect: Excessive overlap creates redundancy without necessarily improving relevance.

Overall explanation

Logical chunking ensures meaningful content is preserved while adhering to token limits, optimizing retrieval quality.

Question 30

A Generative AI Engineer has successfully chunked unstructured documents and wants to store the chunks in a Vector Search index. The current dataframe has columns for the original document name and an array of text chunks. What is the most performant way to store this dataframe?

1. Split the data into train and test sets, create a unique identifier for each document, then save to a Delta table.

Explanation

Incorrect: Splitting into train and test sets is unnecessary for storage in a Vector Search index.

2. Flatten the dataframe to one chunk per row, create a unique identifier for each row, and save to a Delta table.

Explanation

Correct: Flattening ensures efficient retrieval and storage of individual chunks, which is optimal for a Vector Search index.

3. Store each chunk as an independent JSON file in Unity Catalog Volume.

Explanation

Incorrect: JSON storage in Unity Catalog is less performant for large-scale retrieval than Delta tables.

4. First create a unique identifier for each document, then save to a Delta table.

Explanation

Incorrect: This does not address the need to store individual chunks.

Overall explanation

Flattening the dataframe to store individual chunks with unique identifiers optimizes retrieval in a Vector Search index.

Question 31

What is an effective method to preprocess prompts using custom code before sending them to an LLM?

1. It is better not to introduce custom code to preprocess prompts as the LLM has not been trained with examples of the preprocessed prompts

Explanation

Incorrect: Preprocessing prompts is common and does not require retraining the LLM.

2. Directly modify the LLM's internal architecture to include preprocessing steps

Explanation

Incorrect: Modifying the LLM's internal architecture is unnecessary and impractical for preprocessing prompts.

3. Write an MLflow PyFunc model that has a separate function to process the prompts

Explanation

Correct: Using an MLflow PyFunc model allows custom preprocessing of prompts before they are sent to the LLM, ensuring flexibility and task alignment.

4. Rather than preprocessing prompts, it's more effective to postprocess the LLM outputs to align the outputs to desired outcomes

Explanation

Incorrect: While postprocessing is useful, preprocessing ensures the prompt aligns with task requirements before being sent to the LLM.

Overall explanation

A PyFunc model is a robust and scalable solution for preprocessing prompts, allowing custom logic to align inputs with desired outcomes.

Question 32

A Generative AI Engineer is tasked with developing an application that is based on an open-source large language model (LLM). They need a foundation LLM with a large context window. Which model fits this need?

1. MPT-30B

Explanation

Correct: MPT-30B is an open-source LLM with a large context window, making it suitable for handling extensive input text.

2. Llama2-70B

Explanation

Incorrect: Llama2-70B is powerful but does not have as large a context window as MPT-30B.

3. DBRX

Explanation

Incorrect: DBRX is not a known or relevant model for this use case.

4. DistilBERT

Explanation

Incorrect: DistilBERT is a smaller, distilled version of BERT and does not have a large context window.

Overall explanation

MPT-30B is an open-source model optimized for applications requiring a large context window, making it the best choice for this requirement.

Question 33

A Generative AI Engineer is tasked with augmenting user inputs for a chatbot that generates meal plans. The system must use user-provided ingredients and dietary restrictions to recommend meals. How should the input be augmented? **(Choose two)**

1. Include additional context, such as portion sizes and meal preferences, in the prompt.

Explanation

Correct: Contextual details like portion sizes and preferences improve the quality of meal recommendations.

2. Use a classification model to categorize user inputs.

Explanation

Incorrect: Categorization alone does not enhance the chatbot's ability to generate diverse plans.

3. Train a rule-based system for fixed meal plan recommendations.

Explanation

Incorrect: Rule-based systems lack flexibility for dynamic input augmentation.

4. Summarize user inputs to condense key points.

Explanation

Incorrect: Summarization risks omitting critical details needed for meal plans.

5. Add examples of similar meal plans in the prompt.

Explanation

Correct: Examples guide the model to generate more tailored and relevant outputs.

Overall explanation

Contextual prompts and examples ensure the chatbot generates personalized and accurate meal plans based on user inputs.

Question 34

A team wants to serve a code generation model as an assistant for their software developers. It should support multiple programming languages. Quality is the primary objective. Which of the Databricks Foundation Model APIs, or models available in the Marketplace, would be the best fit?

1. MPT-7b

Explanation

Incorrect: MPT-7b is a general-purpose language model with a smaller capacity and is less specialized for code generation.

2. Llama2-70b

Explanation

Incorrect: While Llama2-70b is a powerful general-purpose model, it is not specialized for code generation tasks.

3. BGE-large

Explanation

Incorrect: BGE-large is primarily an embedding model and is not suitable for code generation tasks.

4. CodeLlama-34B

Explanation

Correct: CodeLlama-34B is specifically designed for code generation, supports multiple programming languages, and provides high-quality outputs for developer tasks.

Overall explanation

CodeLlama-34B is a specialized code generation model that supports multiple programming languages, making it the best choice for the team's requirements.

Question 35

A Generative AI Engineer is building a RAG application that will rely on context retrieved from source documents that are currently in PDF format. These PDFs can contain both text and images. They want to develop a solution using the least amount of lines of code. Which Python package should be used to extract the text from the source documents?

1. unstructured

Explanation

Correct: The unstructured Python library is specifically designed for handling PDFs, including those with both text and images, and requires minimal coding effort.

2. beautifulsoup

Explanation

Incorrect: BeautifulSoup is used for parsing HTML and XML documents, not PDFs.

3. numpy

Explanation

Incorrect: Numpy is a numerical computation library and cannot process PDFs.

4. flask

Explanation

Incorrect: Flask is a web framework and cannot extract text from PDFs.

Overall explanation

The unstructured library is designed to efficiently extract text and other structured data from PDFs with minimal code.

Question 36

A Generative AI Engineer is using the code below to test setting up a vector store: `from databricks.vector_search.client import VectorSearchClient vsc = VectorSearchClient() vsc.create_endpoint(name="vector_search_test", endpoint_type="STANDARD")` Assuming they intend to use Databricks-managed embeddings with the default embedding model, what should be the next logical function call?

1. `vsc.get_index()`

Explanation

Incorrect: This retrieves an existing index, but no index has been created yet.

2. `vsc.create_direct_access_index()`

Explanation

Incorrect: While suitable for general use, this option doesn't align with the question's focus on Databricks-managed embeddings.

3. `vsc.create_delta_sync_index()`

Explanation

Correct: This function creates a Delta-synced index, ideal for managing embeddings in Databricks-managed workflows with default settings.

4. `vsc.similarity_search()`

Explanation

Incorrect: Similarity search requires an existing index and cannot be performed at this stage of the workflow.

Overall explanation

Creating a Delta-synced index allows seamless embedding synchronization with Delta tables, aligning with Databricks-managed workflows and default embedding configurations.

Question 37

A Generative AI Engineer has been asked to build an LLM-based question-answering application. The application should take into account new documents that are frequently published. The engineer wants to build this application with the least cost, least development effort, and lowest operational cost. Which combination of chaining components and configuration meets these requirements?

1. For the question-answering application, prompt engineering and an LLM are required to generate answers.

Explanation

Incorrect: This setup would not allow retrieval of up-to-date documents, failing to meet the requirements.

2. For the application a prompt, an agent, and a fine-tuned LLM are required. The agent is used by the LLM to retrieve relevant content that is inserted into the prompt which is given to the LLM to generate answers.

Explanation

Incorrect: Fine-tuning is not needed here and increases development effort unnecessarily.

3. For the application a prompt, a retriever, and an LLM are required. The retriever output is inserted into the prompt which is given to the LLM to generate answers.

Explanation

Correct: This approach ensures the application dynamically retrieves and processes relevant documents with minimal complexity and cost.

4. The LLM needs to be frequently fine-tuned with the new documents in order to provide the most up-to-date answers.

Explanation

Incorrect: Frequent fine-tuning is costly and time-consuming, making it impractical for low-cost applications.

Overall explanation

A prompt, retriever, and LLM setup achieves dynamic responses while minimizing cost and complexity, making it the ideal choice for this scenario.

Question 38

A Generative AI Engineer would like an LLM to generate formatted JSON from emails. This requires parsing and extracting the following information: order ID, date, and sender email.

Here's a sample email: Date: April 23, 2024 Time: 4:22 PM From: anjali.thayer@example.com To: support@company.com Subject: Shipment details Hey there, I have a shipment (order ID is CD34RFT). Can you please send me an update? Thank you, Anjali They need to write a prompt that will extract the relevant information in JSON format with the highest level of output accuracy. Which prompt will achieve this?

1. You will receive customer emails and need to extract date, sender email, and order ID. Return the extracted information in JSON format. Here's an example: {"date": "April 16, 2024", "sender_email": "john.doe@example.com", "order_id": "ORD98765"}.

Explanation

Correct: This prompt includes both clear instructions and an example JSON output, ensuring high accuracy.

2. You will receive customer emails and need to extract date, sender email, and order ID. Return the extracted information in a human-readable format.

Explanation

Incorrect: A human-readable format does not meet the requirement to output data in JSON.

3. You will receive customer emails and need to extract date, sender email, and order ID. You should return the date, sender email, and order ID information in JSON format.

Explanation

Incorrect: While this specifies JSON, it does not provide an example, which may reduce clarity for the LLM.

4. You will receive customer emails and need to extract date, sender email, and order ID. Return the extracted information in JSON format.

Explanation

Incorrect: Although it specifies JSON, the lack of an example can lead to ambiguous or inconsistent outputs.

Overall explanation

Including an example of the desired JSON output format provides explicit guidance to the LLM, ensuring high accuracy and consistency.

Question 39

A Generative AI Engineer is evaluating the quality of translations generated by an LLM-based translation application. Which metric should they use to assess the accuracy of the translations?

1. Perplexity

Explanation

Incorrect: Perplexity measures how well a language model predicts text but does not evaluate translation quality.

2. Cosine Similarity

Explanation

Incorrect: Cosine similarity measures vector similarity and is not a standard metric for evaluating translations.

3. ROUGE

Explanation

Incorrect: ROUGE is primarily used for summarization tasks, not translation.

4. BLEU

Explanation

Correct: BLEU (Bilingual Evaluation Understudy) is a widely used metric for assessing the quality of machine translations by comparing n-gram overlaps with reference translations.

Overall explanation

BLEU is the standard metric for evaluating machine translation quality by measuring how closely the generated text matches reference translations using n-gram comparisons.

Question 40

A Generative AI Engineer is building an LLM application that generates summaries of technical research articles. Which metric should they use to evaluate the quality of the summaries?

1. Word Count

Explanation

Incorrect: Word count does not evaluate quality; it only measures the length of the text.

2. ROUGE

Explanation

Correct: ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is a metric specifically designed to evaluate summarization quality by comparing generated summaries to reference summaries.

3. BLEU

Explanation

Incorrect: BLEU is used for evaluating translation tasks, not summarization.

4. Latency

Explanation

Incorrect: Latency measures response time, not the accuracy or relevance of summaries.

Overall explanation

ROUGE evaluates summarization quality by measuring overlap in recall, precision, and F1 score between generated and reference summaries. It is a standard metric for this use case.

Question 41

A Generative AI Engineer is designing a RAG application to help users answer questions about technical regulations as they learn a new sport. What sequence of steps should they follow to build and deploy this application?

1. Ingest documents from a source → Index the documents and save to Vector Search → User submits queries against an LLM → LLM retrieves relevant documents → Evaluate model → LLM generates a response → Deploy it using Model Serving.

Explanation

Incorrect: This sequence evaluates the model too late in the process, which could lead to deploying an underperforming pipeline.

2. Ingest documents from a source → Index the documents and save to Vector Search → User submits queries against an LLM → LLM retrieves relevant documents → LLM generates a response → Evaluate model → Deploy it using Model Serving.

Explanation

Correct: This sequence follows the proper workflow, ensuring evaluation is completed before final deployment.

3. Ingest documents from a source → Index the documents and save to Vector Search → Evaluate model → Deploy it using Model Serving.

Explanation

Incorrect: This omits crucial steps like query submission and response generation.

4. User submits queries against an LLM → Ingest documents from a source → Index the documents and save to Vector Search → LLM retrieves relevant documents → LLM generates a response → Evaluate model → Deploy it using Model Serving.

Explanation

Incorrect: This sequence is illogical as queries cannot occur before documents are ingested and indexed.

Overall explanation

Following the proper sequence ensures the RAG application is correctly built, tested, and deployed to handle user queries effectively.

Question 42

A Generative AI Engineer is developing a patient-facing healthcare-focused chatbot. If the patient's question is not a medical emergency, the chatbot should solicit more information and suggest relevant articles. If the question is urgent, it should direct the patient to emergency services. Given the input: "I have been experiencing severe headaches and dizziness for the past two days," which response is most appropriate?

1. Here are a few relevant articles for your browsing. Let me know if you have questions after reading them.

Explanation

Incorrect: Providing articles alone does not address the severity of the symptoms.

2. Please provide your age, recent activities, and any other symptoms you have noticed along with your headaches and dizziness.

Explanation

Incorrect: While gathering more details is helpful, it is not appropriate given the potential emergency described.

3. Please call your local emergency services.

Explanation

Correct: Severe headaches and dizziness could indicate a medical emergency, so directing the user to emergency services is the most appropriate response.

4. Headaches can be tough. Hope you feel better soon!

Explanation

Incorrect: This response is too casual and does not address the urgency of the symptoms.

Overall explanation

When symptoms suggest a potential emergency, the chatbot must prioritize safety by directing the user to contact emergency services.

Question 43

A Generative AI Engineer is creating an agent-based LLM system for their favorite monster truck team. The system can answer text-based questions about the team, look up event dates via an API call, or query tables on the team's latest standings. How could the Generative AI Engineer best design these capabilities into their system?

1. Instruct the LLM to respond with "RAG," "API," or "TABLE" depending on the query, then use text parsing and conditional statements to resolve the query.

Explanation

Incorrect: This method lacks scalability and doesn't align with agent-based design.

2. Build a system prompt with all possible event dates and table information in the system prompt. Use a RAG architecture to lookup generic text questions and otherwise leverage the information in the system prompt.

Explanation

Incorrect: Including all information in the system prompt reduces efficiency and scalability.

3. Write a system prompt for the agent listing available tools and bundle it into an agent system that runs a number of calls to solve a query.

Explanation

Correct: Defining tools for the agent and using them to process different queries ensures the system can handle diverse question types effectively.

4. Ingest PDF documents about the monster truck team into a vector store and query it in a RAG architecture.

Explanation

Incorrect: This approach only handles text-based questions and doesn't address API or table queries.

Overall explanation

Using an agent with tools for handling text queries, API lookups, and table queries ensures flexibility and scalability for diverse system requirements.

Question 44

A Generative AI Engineer has been asked to design an LLM-based application that answers employee HR questions using HR PDF documentation. Which set of high-level tasks should the Generative AI Engineer's system perform?

1. Create an interaction matrix of historical employee questions and HR documentation. Use ALS to factorize the matrix and create embeddings. Calculate the embeddings of new queries and use them to find the best HR documentation. Use an LLM to generate a response to the employee question based upon the documentation retrieved.

Explanation

Creating an interaction matrix of historical employee questions and HR documentation and using ALS to factorize the matrix may not be the most effective approach for generating responses to employee HR questions. This method may not capture the nuances and context required for accurate responses.

2. Use an LLM to summarize HR documentation. Provide summaries of documentation and user query into an LLM with a large context window to generate a response to the user.

Explanation

Using an LLM to summarize HR documentation may not provide accurate responses to employee HR questions, as the summarization may lose important details and context from the original documentation.

Providing summaries of documentation and user queries directly to an LLM may not yield the desired results.

3. Calculate averaged embeddings for each HR document, compare embeddings to user query to find the best document. Pass the best document with the user query into an LLM with a large context window to generate a response to the employee.

Explanation

Calculating averaged embeddings for each HR document and comparing them to the user query is not an efficient way to find the best document. Additionally, using an LLM with a large context window may not be necessary for generating responses to HR questions based on PDF documentation.

4. Split HR documentation into chunks and embed into a vector store. Use the employee question to retrieve best-matched chunks of documentation and use the LLM to generate a response to the employee based upon the documentation retrieved.

Explanation

Splitting HR documentation into chunks and embedding them into a vector store allows for efficient retrieval of relevant information based on employee questions. Using the LLM to generate responses based on the retrieved documentation chunks ensures that the responses are contextually relevant and accurate, making this approach the most suitable for the task at hand.

Overall explanation

Chunking HR documents and embedding them into a vector store ensures accurate retrieval and relevance for employee questions, making it the best solution.

Question 45

A Generative AI Engineer deployed an application but found that request volume is insufficient for provisioned throughput. How can they ensure cost-effectiveness for deployment?

1. Change to a model with fewer parameters to reduce hardware constraints.

Explanation

Incorrect: Reducing model size may lower cost but does not directly address throughput-related cost issues.

2. Deploy the model using pay-per-token throughput as it comes with cost guarantees.

Explanation

Correct: Pay-per-token ensures costs scale with usage, making it cost-effective for low request volumes.

3. Throttle incoming request batches manually to avoid rate-limiting issues.

Explanation

Incorrect: Throttling helps manage requests but does not optimize cost-effectiveness.

4. Switch to using External Models instead.

Explanation

Incorrect: External models do not address cost-effectiveness for request throughput.

Overall explanation

Pay-per-token throughput ensures that costs align with usage, making it the best strategy for low request volumes.